

Bài 4:

Huấn luyện mạng nơ-ron

Nội dung

- Hàm kích hoạt
- Tiền xử lý dữ liệu
- Khởi tạo trọng số
- Các kỹ thuật chuẩn hóa

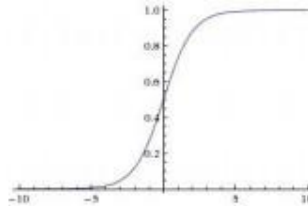
Hàm kích hoạt

Hàm kích hoạt

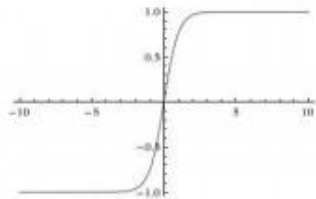
Activation Functions

Sigmoid

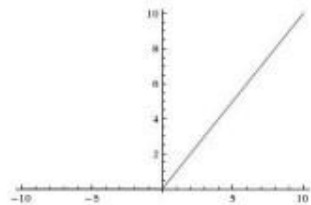
$$\sigma(x) = 1/(1 + e^{-x})$$



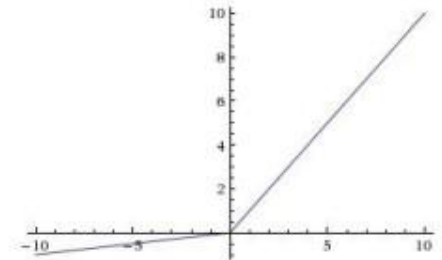
tanh tanh(x)



ReLU max(0,x)



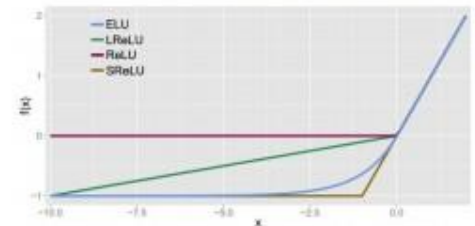
Leaky ReLU $\max(0.1x, x)$



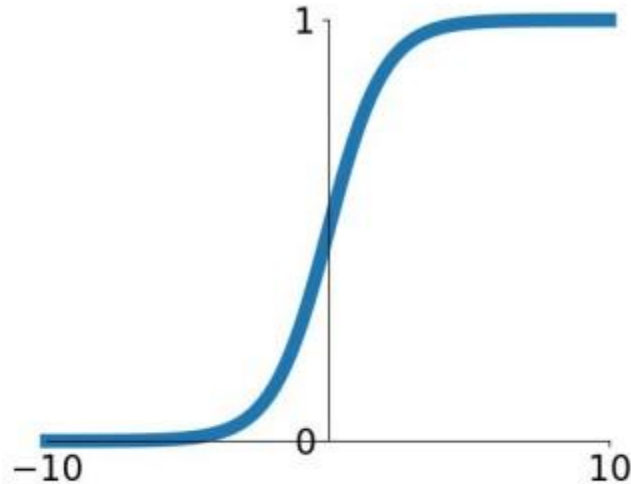
Maxout $\max(w_1^T x + b_1, w_2^T x + b_2)$

ELU

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha (\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$



Hàm kích hoạt



Sigmoid

$$\sigma(x) = 1 / (1 + e^{-x})$$

- Nhận giá trị trong khoảng $[0, 1]$
- Được dùng phổ biến trong lịch sử mạng nơ-ron do chúng mô phỏng tốt tỉ lệ bắn xung (firing rate) của nơ-ron
- Có 3 nhược điểm:
 - Nơ-ron bão hòa triệt tiêu gradient

Hàm kích hoạt

- Nơ-ron bão hòa triệt tiêu gradient là một khái niệm trong học máy và học sâu.
- Để bạn dễ hiểu hơn, hãy tưởng tượng bạn đang leo lên một ngọn đồi rất dốc, và càng lên cao, bạn càng gặp khó khăn trong việc giữ thăng bằng vì độ dốc càng lớn.
- Trong quá trình học sâu, gradient đại diện cho độ dốc này và giúp điều chỉnh các trọng số trong mạng nơ-ron.

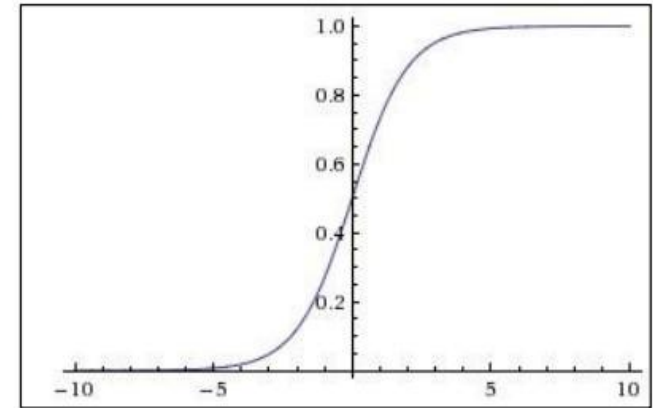
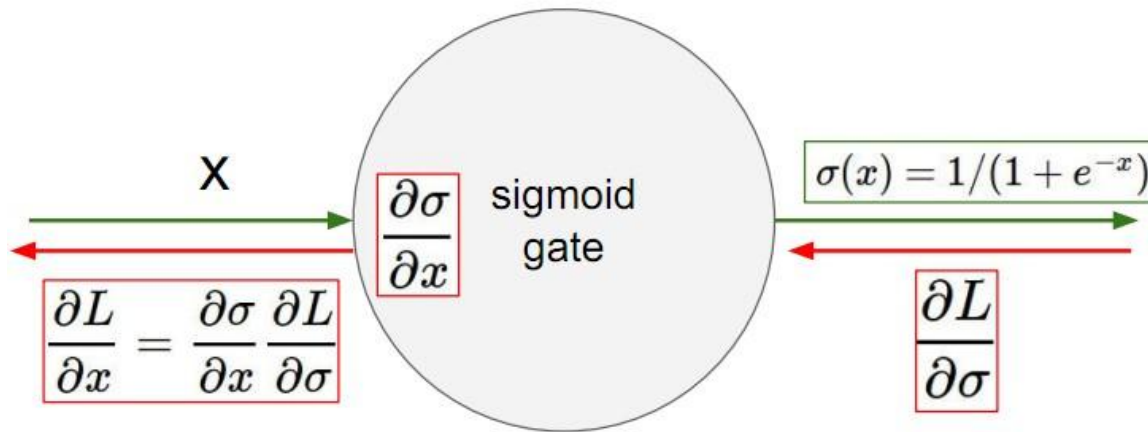
Hàm kích hoạt

- Khi nơ-ron bị bão hòa, gradient trở nên rất nhỏ hoặc thậm chí bằng không, khiến cho quá trình điều chỉnh trọng số gặp khó khăn.
- Điều này làm cho mạng học rất chậm hoặc ngừng học hoàn toàn, vì không có đủ "lực đẩy" để vượt qua những chỗ khó khăn.
- Đây là một vấn đề phổ biến, đặc biệt trong các mạng nơ-ron sâu.

Hàm kích hoạt

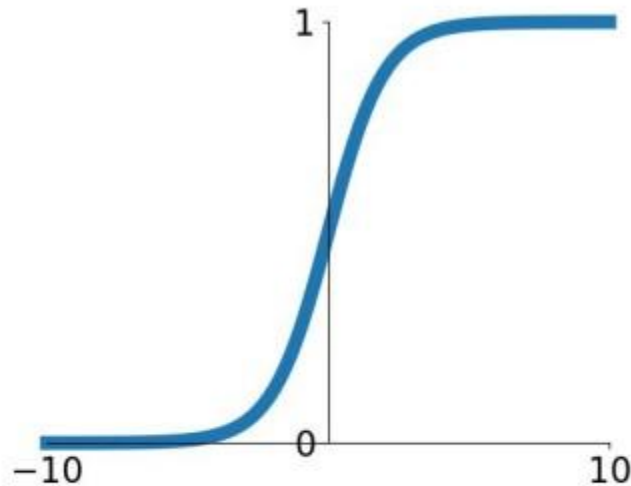
- Hàm sigmoid biến đổi giá trị đầu vào thành một giá trị trong khoảng từ 0 đến 1. Khi giá trị đầu vào rất lớn hoặc rất nhỏ, đầu ra của hàm sigmoid tiến gần đến 1 hoặc 0. Tại các giá trị này, gradient của hàm sigmoid tiến gần đến 0.
- Vì gradient đại diện cho tốc độ thay đổi của hàm kích hoạt, khi gradient bằng không hoặc rất nhỏ, quá trình học của mạng nơ-ron gặp khó khăn vì không có đủ tín hiệu để điều chỉnh trọng số. Do đó, các nơ-ron dễ dàng rơi vào trạng thái bão hòa và không còn học được gì thêm nữa..

Hàm kích hoạt



- Điều gì sẽ xảy ra khi $x = -10$?
- Điều gì sẽ xảy ra khi $x = 0$?
- Điều gì sẽ xảy ra khi $x = 10$?

Hàm kích hoạt



Sigmoid

$$\sigma(x) = 1 / (1 + e^{-x})$$

- Nhận giá trị trong khoảng $[0, 1]$
- Được dùng phổ biến trong lịch sử mạng nơ-ron do chúng mô phỏng tốt tỉ lệ bắn xung (firing rate) của nơ-ron
- Có 3 nhược điểm:
 - Nơ-ron bão hòa triệt tiêu gradient
 - Trung bình đầu ra khác 0

Hàm kích hoạt

Hàm kích hoạt có phân bố đầu ra với trung bình bằng 0 có ý nghĩa quan trọng trong học máy và học sâu.

Khi đầu ra của hàm kích hoạt có trung bình bằng 0, điều này giúp cân bằng thông tin truyền qua các lớp của mạng nơ-ron, giảm thiểu sự lệch lạc và cải thiện hiệu quả của quá trình huấn luyện. Dưới đây là một số lợi ích chính:

1. Cân bằng gradient: Khi đầu ra của hàm kích hoạt có trung bình bằng 0, gradient của các lớp trong mạng nơ-ron sẽ được cân bằng hơn. Điều này giúp tránh hiện tượng gradient biến mất hoặc bùng nổ, cải thiện khả năng hội tụ trong quá trình huấn luyện.

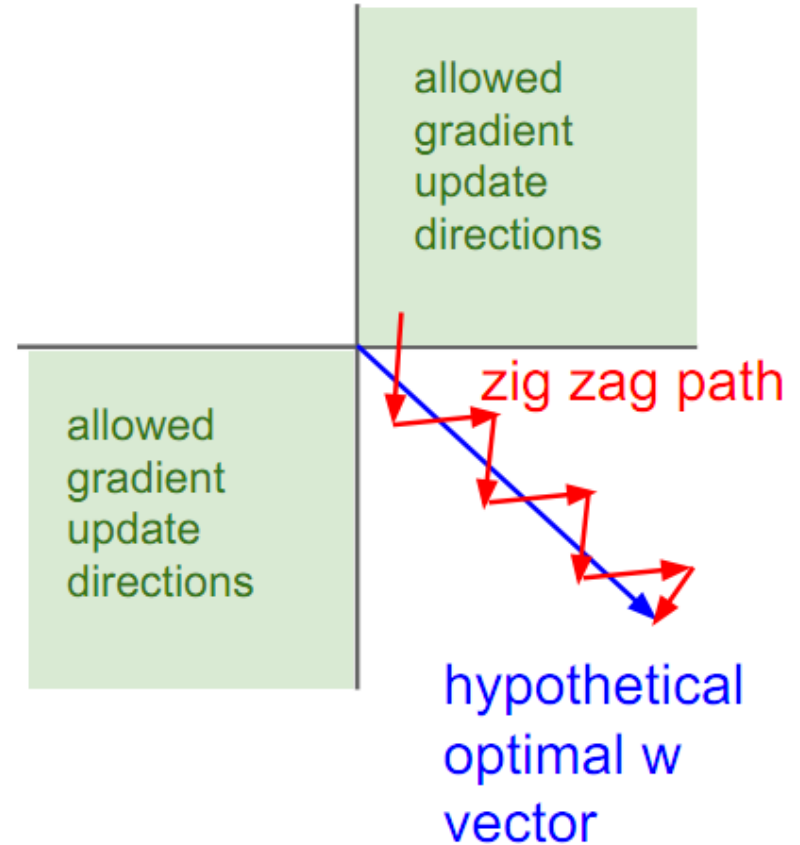
2. Tăng tốc độ học: Các hàm kích hoạt có trung bình đầu ra bằng 0 giúp tăng tốc độ học vì không có sự lệch lạc lớn trong thông tin truyền qua các lớp. Điều này giúp trọng số trong mạng nơ-ron được điều chỉnh hiệu quả hơn.

3. Giảm sự lệch lạc: Khi đầu ra của các nơ-ron không bị lệch về một phía (tức là trung bình bằng 0), mạng nơ-ron sẽ không bị lệch lạc và hoạt động hiệu quả hơn trong việc biểu diễn các hàm phức tạp.

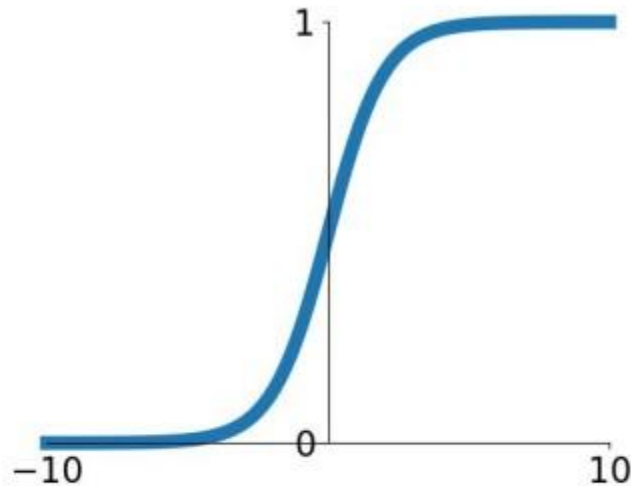
Hàm kích hoạt

$$f\left(\sum_i w_i x_i + b\right)$$

- Điều gì xảy ra nếu tất cả đầu vào x_i của nơ-ron đều dương?
- Khi đó gradient của hàm mục tiêu đối với w sẽ ra sao?
- Tất cả các phần tử của w đều cùng dấu với $f'(w)$, tức là cùng âm hoặc cùng dương
- Khi đó gradient chỉ có thể hướng theo một số chiều nhất định trong không gian tìm kiếm



Hàm kích hoạt

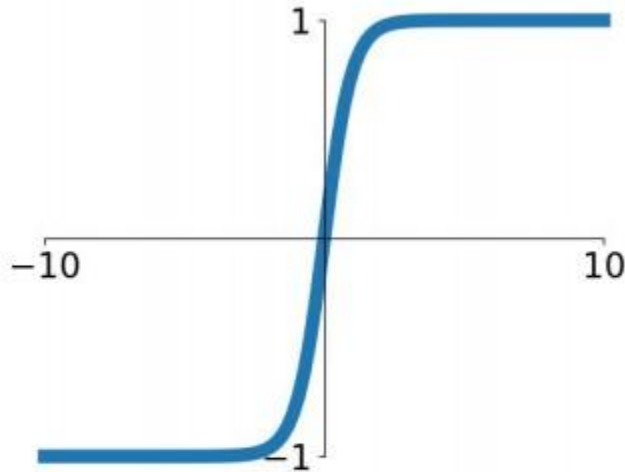


Sigmoid

$$\sigma(x) = 1 / (1 + e^{-x})$$

- Nhận giá trị trong khoảng $[0, 1]$
- Được dùng phổ biến trong lịch sử mạng nơ-ron do chúng mô phỏng tốt tỉ lệ bắn xung (firing rate) của nơ-ron
- Có 3 nhược điểm:
 - Nơ-ron bão hòa triệt tiêu gradient
 - Trung bình đầu ra khác 0
 - Tính toán hàm mũ $\exp()$ tốn kém

Hàm kích hoạt

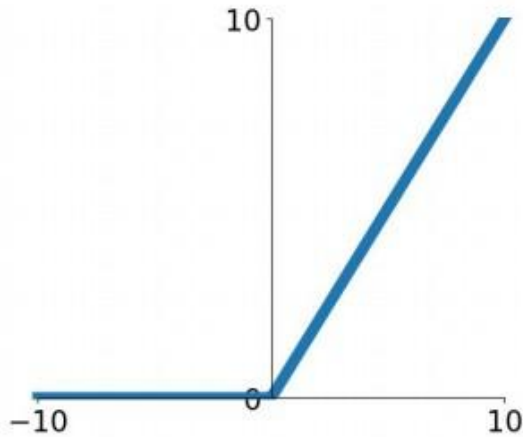


tanh(x)

$$\frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Nhận giá trị trong khoảng $[-1, 1]$
- Trung bình đầu ra bằng 0
- Vẫn bị hiện tượng bão hòa, triệt tiêu gradient

Hàm kích hoạt

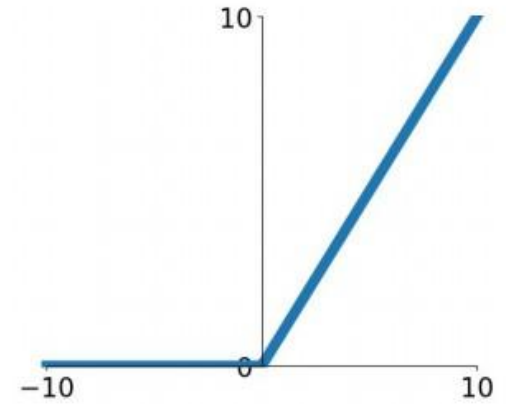
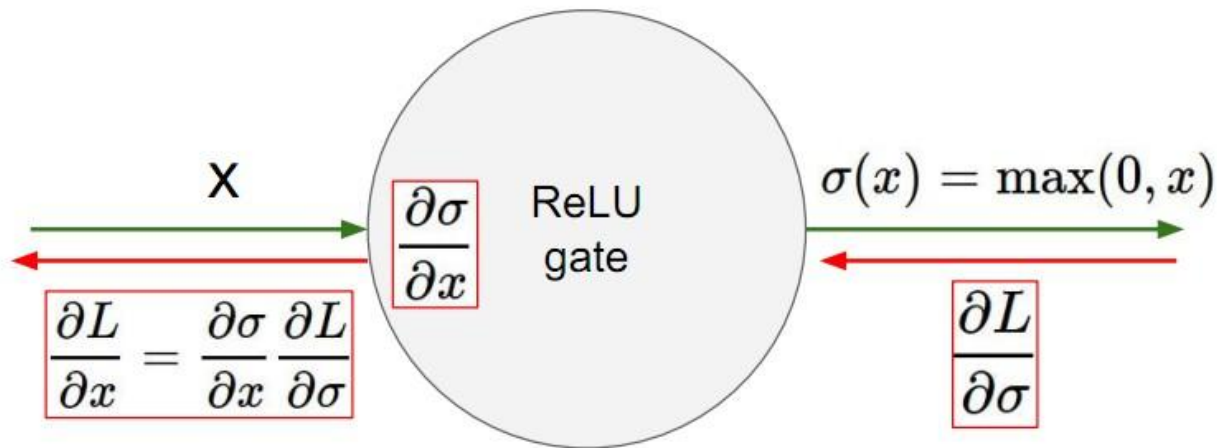


ReLU
(Rectified Linear Unit)

$$f(x) = \max(0, x)$$

- Không bị bão hòa trong vùng dương
 - Tính toán hiệu quả
 - Trong thực tế hội tụ nhanh hơn sigmoid/tanh (khoảng 6 lần)
- Đầu ra trung bình khác 0
 - Và một vấn đề nữa...

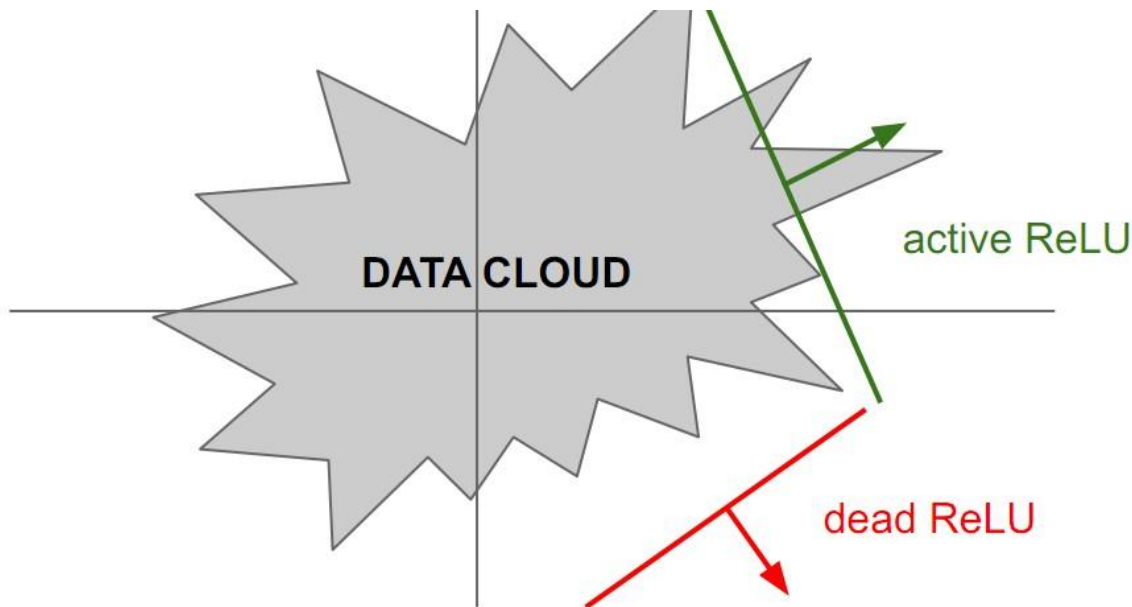
Hàm kích hoạt



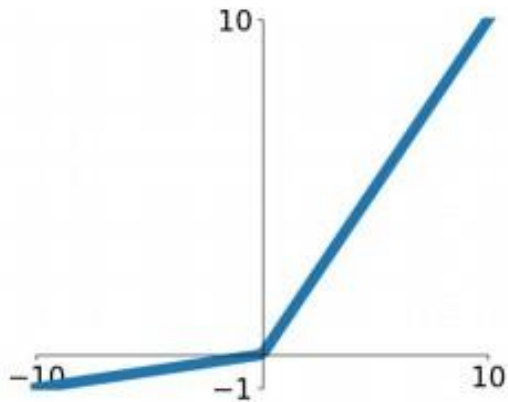
- Điều gì sẽ xảy ra khi $x = -10$?
- Điều gì sẽ xảy ra khi $x = 0$?
- Điều gì sẽ xảy ra khi $x = 10$?

Hàm kích hoạt

- ReLU bị “văng” ra khỏi tập dữ liệu dẫn tới đầu ra luôn âm và không bao giờ được cập nhật trọng số nữa
→ ReLU chết
- Thường khởi tạo nơ-ron ReLU với bias dương bé (ví dụ 0.01)



Hàm kích hoạt

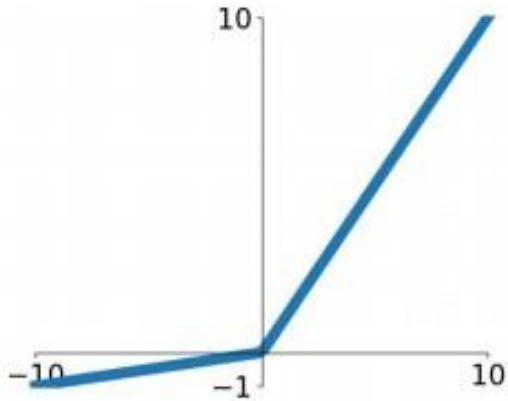


Leaky ReLU

$$f(x) = \max(0.01x, x)$$

- Không bị bão hòa trong vùng dương
- Tính toán hiệu quả
- Trong thực tế hội tụ nhanh hơn sigmoid/tanh (khoảng 6 lần)
- Không bao giờ “chết”

Hàm kích hoạt



Leaky ReLU

$$f(x) = \max(0.01x, x)$$

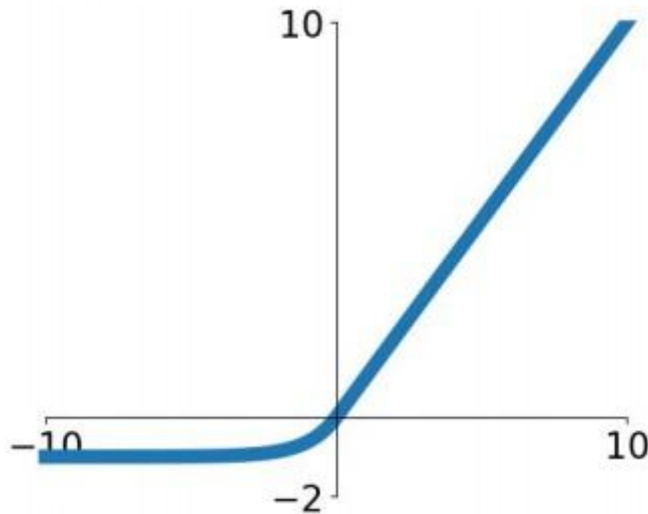
- Không bị bão hòa trong vùng dương
- Tính toán hiệu quả
- Trong thực tế hội tụ nhanh hơn sigmoid/tanh (khoảng 6 lần)
- Không bao giờ “chết”

Parametric Rectifier (PReLU)

$$f(x) = \max(\alpha x, x)$$

backprop into α
(parameter)

Hàm kích hoạt ELU



- Có tất cả ưu điểm của ReLU
- Trung bình đầu ra gần 0 hơn
- Không “chết”
- Tính toán lâu do có hàm $\exp()$

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha (\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

Hàm kích hoạt Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

- Tổng quát hóa của ReLU và Leaky ReLU
- Tính toán tuyến tính
- Không bão hòa
- Không chết
- Gấp đôi số tham số mỗi nơ-ron

Hàm kích hoạt

- **Trong thực tế:**

- Thường dùng ReLU. Cần thận với tốc độ học để tránh ReLU bị chết.
- Có thể thử Leaky ReLU / Maxout / ELU
- Có thể thử tanh nhưng không kỳ vọng nhiều
- Không dùng sigmoid

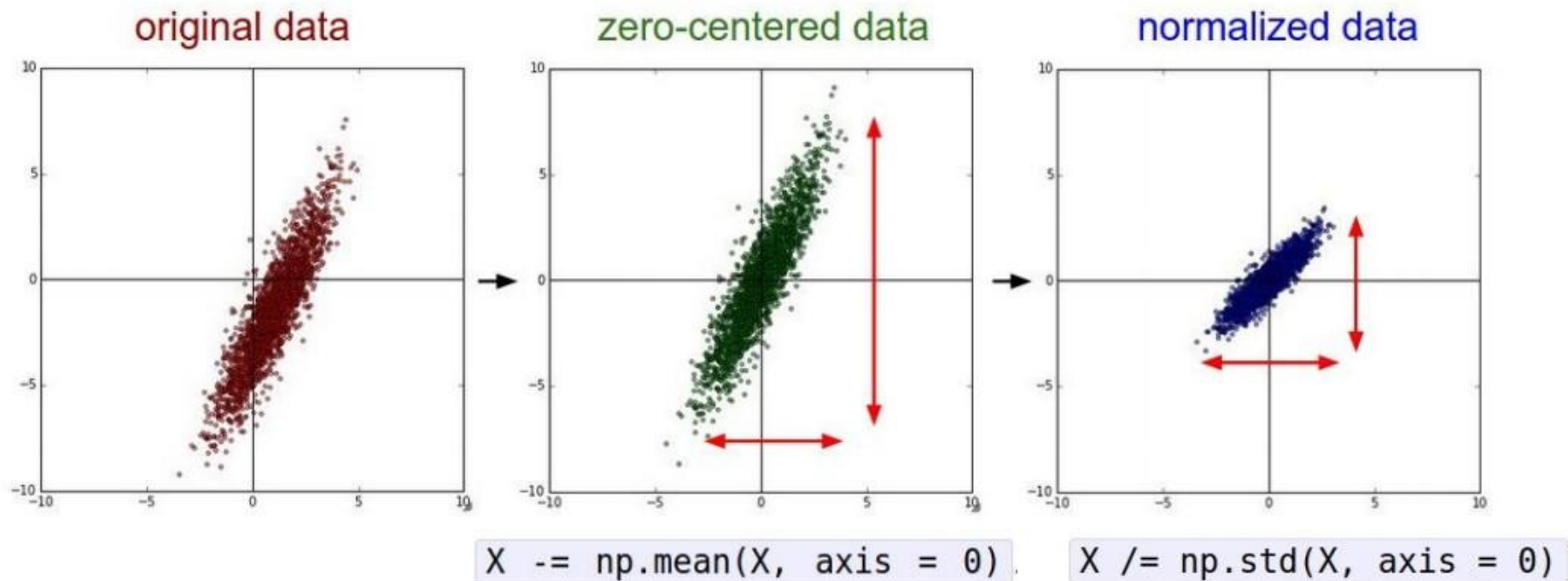
- Gần đây xuất hiện một số hàm kích hoạt mới:

- $\text{ReLU6} = \min(6, \text{ReLU}(x))$
- Swish
- Mish

Tiền xử lý dữ liệu

Tiền xử lý dữ liệu

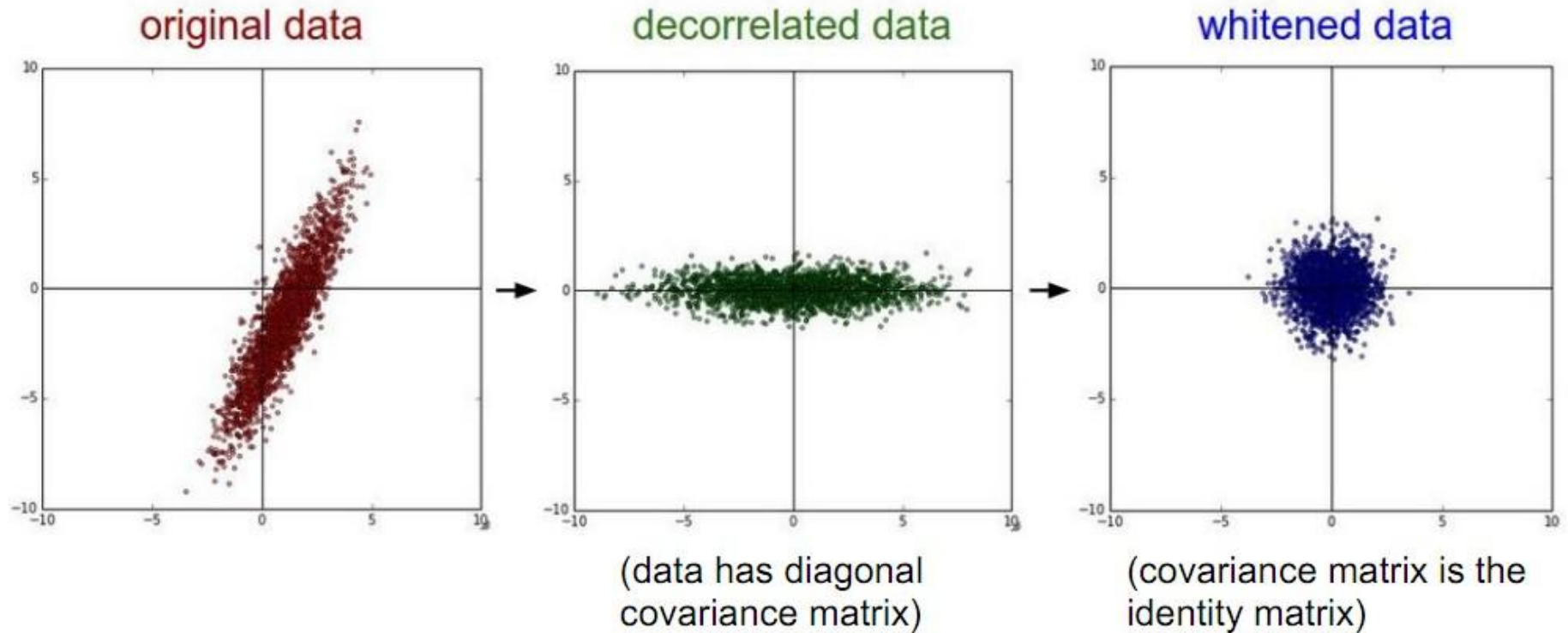
- Biến đổi phân phối dữ liệu về kỳ vọng bằng 0: trừ tất cả mẫu dữ liệu cho mẫu trung bình
- Biến đổi phân phối dữ liệu về độ lệch chuẩn đơn vị



Giả sử X [NxD] là ma trận dữ liệu, mỗi mẫu dữ liệu là một dòng

Tiền xử lý dữ liệu

- Trong thực tế có thể sử dụng PCA (Principal Component Analysis) hoặc Whitening dữ liệu



Tiền xử lý dữ liệu

PCA (Principal Component Analysis) là một kỹ thuật thống kê quan trọng được sử dụng để giảm số chiều của dữ liệu mà không làm mất đi quá nhiều thông tin. Mục đích của PCA là biến đổi một tập hợp các biến có tương quan với nhau thành một tập hợp các biến không tương quan (các thành phần chính).

Dưới đây là các bước cơ bản khi sử dụng PCA:

- 1. Chuẩn hóa dữ liệu:** Điều này có nghĩa là điều chỉnh các biến để chúng có cùng tầm quan trọng, vì PCA rất nhạy cảm với độ lệch của dữ liệu.
- 2. Tính ma trận hiệp phương sai:** Để hiểu mối quan hệ giữa các biến.
- 3. Tính toán các vector riêng và giá trị riêng:** Từ ma trận hiệp phương sai để xác định các thành phần chính.
- 4. Chọn các thành phần chính:** Giữ lại một số ít các thành phần chính dựa trên mức độ quan trọng của chúng.
- 5. Chuyển đổi dữ liệu:** Sử dụng các thành phần chính để chuyển đổi dữ liệu gốc thành tập dữ liệu mới với số chiều ít hơn.

Tiền xử lý dữ liệu

Whitening là một quá trình chuyển đổi các dữ liệu đầu vào sao cho các biến số có trung bình bằng 0 và phương sai bằng 1. Mục đích chính của Whitening là loại bỏ sự tương quan giữa các biến, làm cho dữ liệu trở nên trực giao (không phụ thuộc lẫn nhau), giúp các thuật toán học máy hoạt động hiệu quả hơn.

Dưới đây là các bước cơ bản khi sử dụng Whitening:

- 1.Chuẩn hóa dữ liệu:** Trước tiên, cần chuẩn hóa dữ liệu sao cho mỗi biến có trung bình bằng 0 và phương sai bằng 1.
- 2.Tính ma trận hiệp phương sai:** Xác định mối quan hệ giữa các biến.
- 3.Phân tích giá trị riêng và vector riêng:** Từ ma trận hiệp phương sai để tìm các hướng chính của dữ liệu.
- 4.Chuyển đổi dữ liệu:** Sử dụng các giá trị riêng và vector riêng để chuyển đổi dữ liệu gốc thành dữ liệu trắng (whitened data).

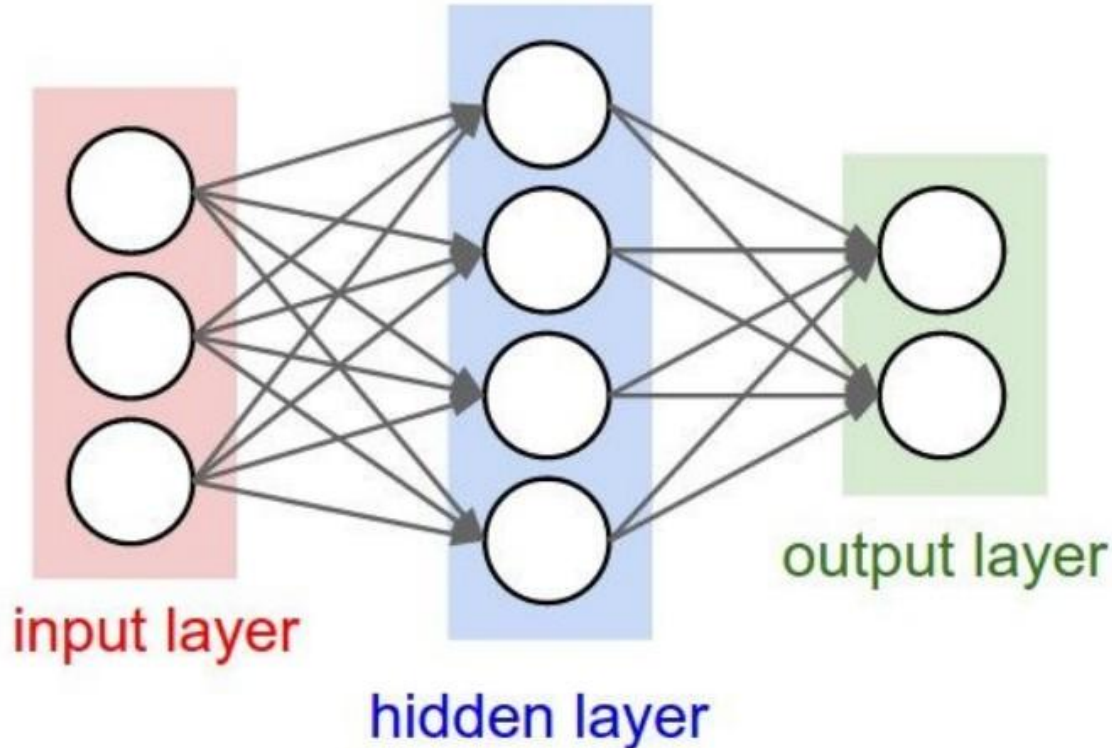
Tiền xử lý dữ liệu

- Ví dụ với bộ CIFAR10 với các ảnh kích thước 32x32x3
 - Subtract the mean image (e.g. AlexNet)
(mean image = [32,32,3] array)
 - Subtract per-channel mean (e.g. VGGNet)
(mean along each channel = 3 numbers)
 - Subtract per-channel mean and
Divide by per-channel std (e.g. ResNet)
(mean along each channel = 3 numbers)
- Thường ít sử dụng PCA hoặc whitening

Khởi tạo trọng số

Khởi tạo trọng số

- Điều gì xảy ra nếu khởi tạo tất cả các trọng số bằng 0?
- Không có ý nghĩa do tất cả các nơ-ron đều học và xử lý giống hệt nhau



Khởi tạo trọng số

- Ý tưởng thứ nhất: Khởi tạo ngẫu nhiên các giá trị nhỏ (Ví dụ theo phân bố chuẩn với kỳ vọng 0, độ lệch chuẩn 0.01)

```
W = 0.01 * np.random.randn(Din, Dout)
```

Làm việc ổn với các mạng nơ-ron nhỏ, nhưng có vấn đề với các mạng nơ-ron sâu hơn.

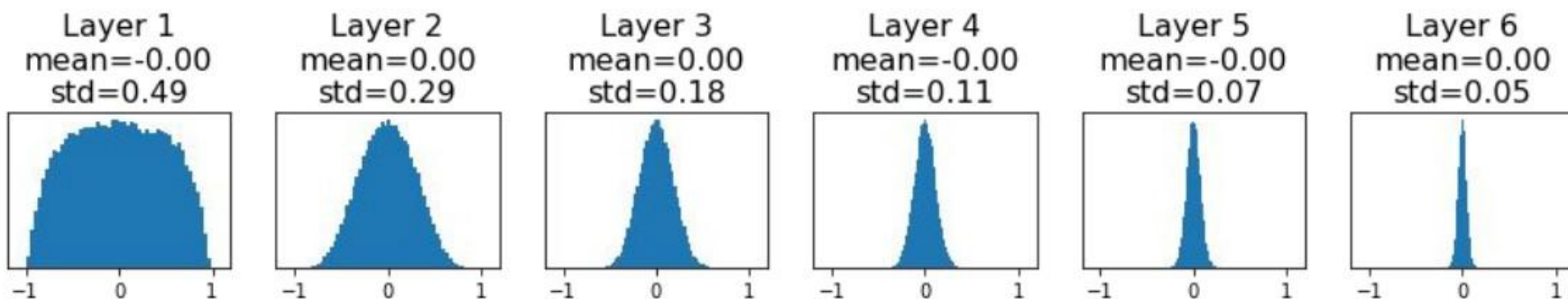
Khởi tạo trọng số

```
dims = [4096] * 7      Forward pass for a 6-layer
hs = []                net with hidden size 4096
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

All activations tend to zero for deeper network layers

Q: What do the gradients dL/dW look like?

A: All zero, no learning =(



- Gradient $dL/dW = 0$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l$$

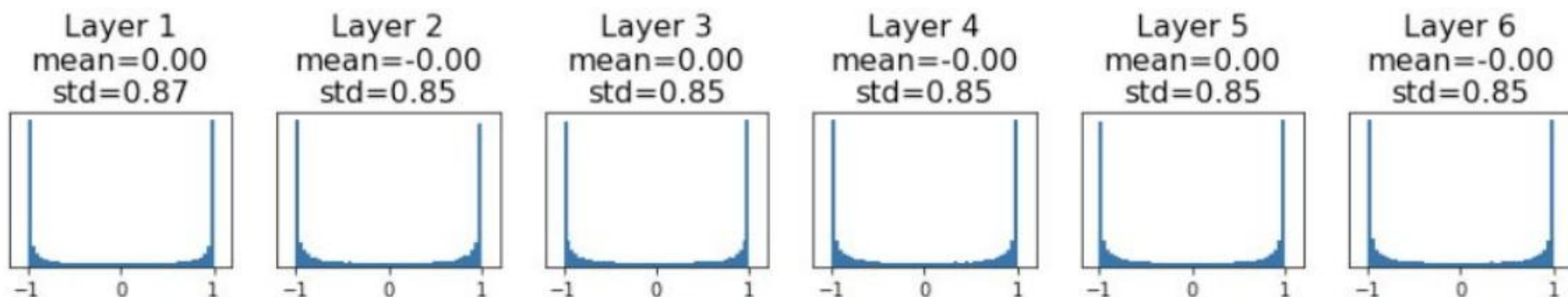
$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l)$$

Khởi tạo trọng số

```
dims = [4096] * 7    Increase std of initial
hs = []              weights from 0.01 to 0.05
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

All activations saturate

Q: What do the gradients look like?



- Gradient $dL/dW = 0$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l$$

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l)$$

Khởi tạo trọng số: Khởi tạo Xavier

- Giả sử x và w là iid, độc lập nhau và trung bình bằng 0
- Tính toán theo chiều tiến forward:

$$\text{var}(y) = \text{var}(w_1x_1 + w_2x_2 + \dots + w_{N_{in}}x_{N_{in}} + b)$$

$$\text{var}(w_ix_i) = E(x_i)^2\text{var}(w_i) + E(w_i)^2\text{var}(x_i) + \text{var}(w_i)\text{var}(x_i)$$

$$\text{var}(y) = N_{in} * \text{var}(w_i) * \text{var}(x_i)$$

$$N_{in} * \text{var}(w_i) = 1$$

$$\text{var}(w_i) = 1 / N_{in}$$

- Tương tự với luồng tín hiệu gradient backward:

$$\text{var}(w_i) = 1 / N_{out}$$

- Trung bình:

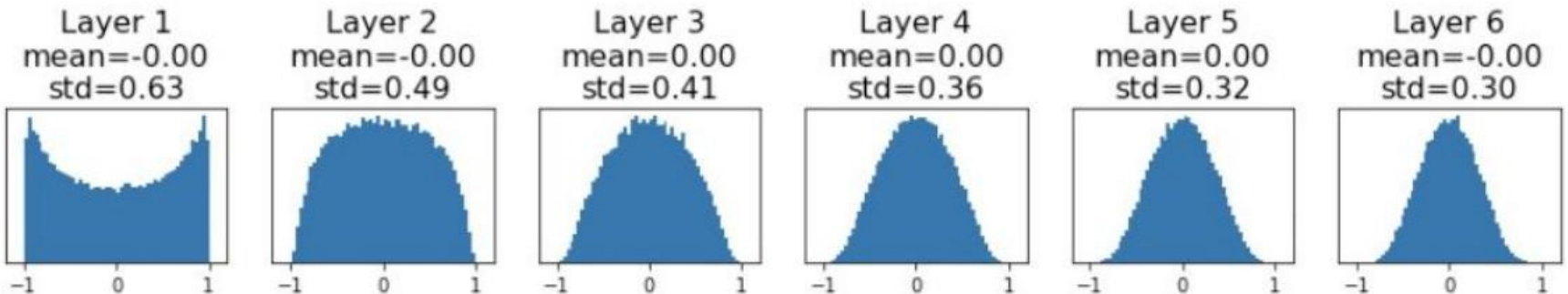
$$\text{var}(w_i) = 2 / (N_{in} + N_{out})$$

Khởi tạo trọng số

- Khởi tạo Xavier

```
dims = [4096] * 7          "Xavier" initialization:
hs = []                    std = 1/sqrt(Din)
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Just right”: Activations are nicely scaled for all layers!

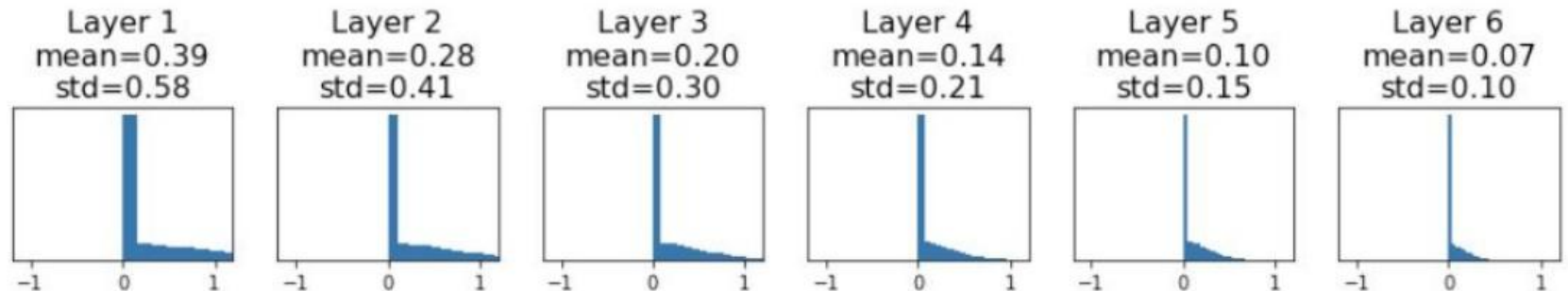


Khởi tạo trọng số

```
dims = [4096] * 7      Change from tanh to ReLU
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

Xavier assumes zero centered activation function

Activations collapse to zero again, no learning =(



$$\text{var}(y) = \text{var}(w_1x_1 + w_2x_2 + \dots + w_{N_{in}}x_{N_{in}} + b)$$

$$\text{var}(y) = N_{in} / 2 * \text{var}(w_i) * \text{var}(x_i)$$

$$N_{in} / 2 * \text{var}(w_i) = 1$$

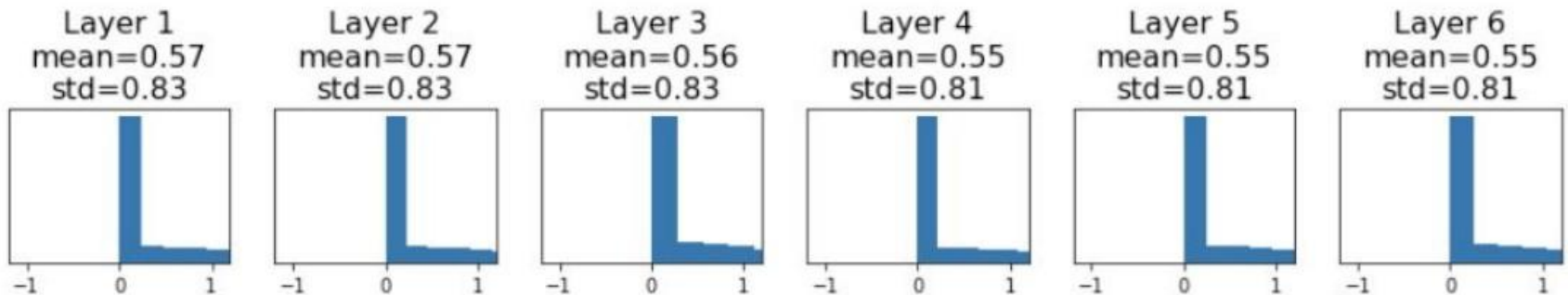
$$\text{var}(w_i) = 2 / N_{in}$$

Khởi tạo trọng số

- He / MSRA Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2/Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

“Just right”: Activations are nicely scaled for all layers!



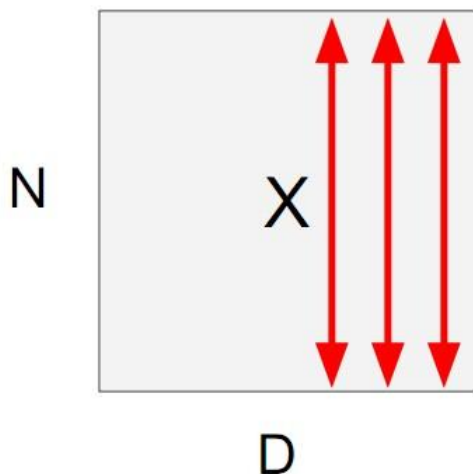
Các kỹ thuật chuẩn hóa

Batch Normalization

- Muốn hàm kích hoạt có phân bố đầu ra với trung bình bằng 0 và độ lệch chuẩn đơn vị? Hãy biến đổi theo ý tưởng đó!

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Input: $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{Per-channel mean, shape is D}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{Per-channel var, shape is D}$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \quad \text{Normalized x, Shape is N x D}$$

Batch Normalization

- Ràng buộc kỳ vọng bằng 0 và độ lệch chuẩn đơn vị là quá chặt! Có thể khiến mô hình bị underfitting.
- ➔ Nới lỏng cho mô hình, tạo lối thoát cho mô hình nếu nó không muốn bị ràng buộc.

Input: $x : N \times D$

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{Per-channel mean, shape is D}$$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{Per-channel var, shape is D}$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}} \quad \text{Normalized x, Shape is N x D}$$

Learning $\gamma = \sigma$,
 $\beta = \mu$, will recover the identity function!

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j \quad \text{Output, Shape is N x D}$$

Batch Normalization

- Không thể tính kỳ vọng và phương sai theo lô dữ liệu (batch) lúc suy diễn

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$,
 $\beta = \mu$, will recover the identity function!

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{Per-channel mean, shape is D}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{Per-channel var, shape is D}$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}} \quad \text{Normalized x, Shape is N x D}$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j \quad \text{Output, Shape is N x D}$$

Batch Normalization

- Lúc suy diễn, BN đơn giản là phép biến đổi tuyến tính.
Có thể áp dụng phía sau lớp FC hoặc conv

Input: $x : N \times D$

$\mu_j =$ (Running) average of
values seen during training

Per-channel mean,
shape is D

**Learnable scale and
shift parameters:**

$\gamma, \beta : D$

$\sigma_j^2 =$ (Running) average of
values seen during training

Per-channel var,
shape is D

During testing batchnorm
becomes a linear operator!
Can be fused with the previous
fully-connected or conv layer

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x,
Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output,
Shape is N x D

Batch Normalization

Batch Normalization for
fully-connected networks

$\mathbf{x}: \mathbf{N} \times \mathbf{D}$

Normalize



$\boldsymbol{\mu}, \boldsymbol{\sigma}: 1 \times \mathbf{D}$

$\gamma, \beta: 1 \times \mathbf{D}$

$$\mathbf{y} = \gamma(\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \beta$$

Batch Normalization for
convolutional networks
(Spatial Batchnorm, BatchNorm2D)

$\mathbf{x}: \mathbf{N} \times \mathbf{C} \times \mathbf{H} \times \mathbf{W}$

Normalize

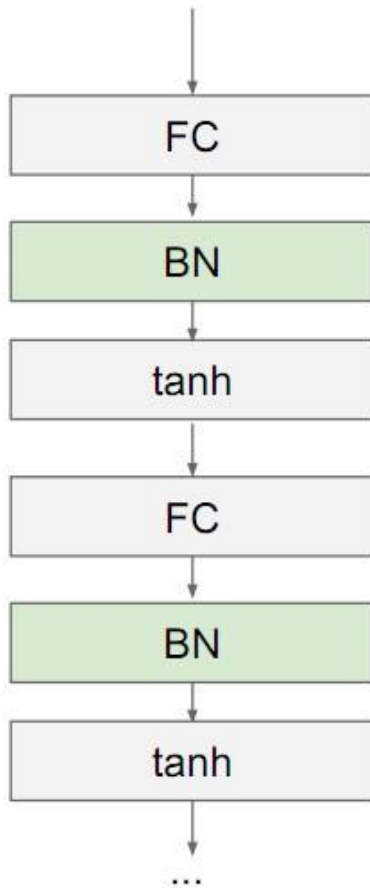


$\boldsymbol{\mu}, \boldsymbol{\sigma}: 1 \times \mathbf{C} \times 1 \times 1$

$\gamma, \beta: 1 \times \mathbf{C} \times 1 \times 1$

$$\mathbf{y} = \gamma(\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \beta$$

Batch Normalization



Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Ưu điểm của BN

- Dễ dàng hơn khi huấn luyện các mạng sâu!
- Cải thiện luồng gradient
- Cho phép huấn luyện với tốc độ học cao hơn, hội tụ nhanh hơn
- Mạng ổn định hơn, đỡ phụ thuộc hơn với khởi tạo trọng số
- Một kiểu ràng buộc khi huấn luyện
- Khi suy diễn không cần tính toán thêm, đơn giản là biến đổi tuyến tính
- Khi huấn luyện và khi suy diễn làm việc khác nhau: đây là nguồn gốc gây ra nhiều lỗi!

Chuẩn hóa theo lớp

Batch Normalization for
fully-connected networks

$$\begin{array}{l} \mathbf{x}: \mathbf{N} \times \mathbf{D} \\ \text{Normalize} \downarrow \\ \boldsymbol{\mu}, \boldsymbol{\sigma}: \mathbf{1} \times \mathbf{D} \\ \boldsymbol{\gamma}, \boldsymbol{\beta}: \mathbf{1} \times \mathbf{D} \\ \mathbf{y} = \boldsymbol{\gamma}(\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta} \end{array}$$

Layer Normalization for
fully-connected networks
Same behavior at train and test!
Can be used in recurrent networks

$$\begin{array}{l} \mathbf{x}: \mathbf{N} \times \mathbf{D} \\ \text{Normalize} \downarrow \\ \boldsymbol{\mu}, \boldsymbol{\sigma}: \mathbf{N} \times \mathbf{1} \\ \boldsymbol{\gamma}, \boldsymbol{\beta}: \mathbf{1} \times \mathbf{D} \\ \mathbf{y} = \boldsymbol{\gamma}(\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta} \end{array}$$

Chuẩn hóa theo mẫu

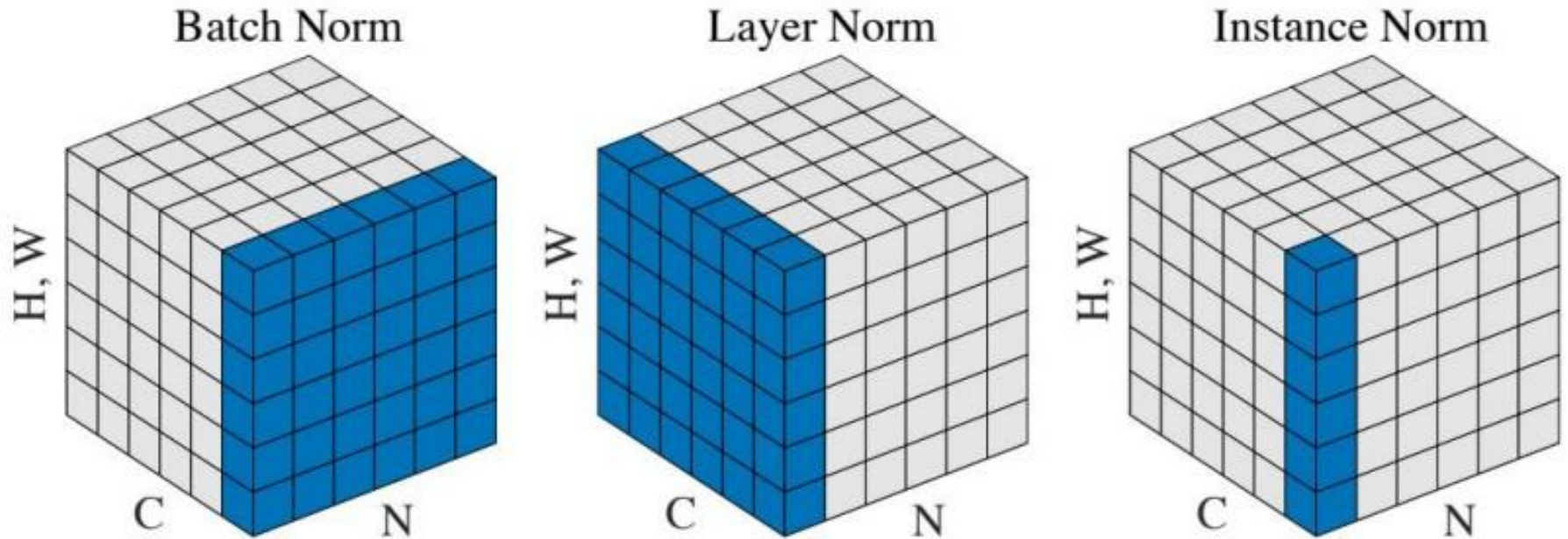
Batch Normalization for
convolutional networks

$$\begin{array}{l} \mathbf{x}: \mathbf{N} \times \mathbf{C} \times \mathbf{H} \times \mathbf{W} \\ \text{Normalize} \quad \downarrow \quad \downarrow \quad \downarrow \\ \boldsymbol{\mu}, \boldsymbol{\sigma}: \mathbf{1} \times \mathbf{C} \times \mathbf{1} \times \mathbf{1} \\ \gamma, \beta: \mathbf{1} \times \mathbf{C} \times \mathbf{1} \times \mathbf{1} \\ \mathbf{y} = \gamma(\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \beta \end{array}$$

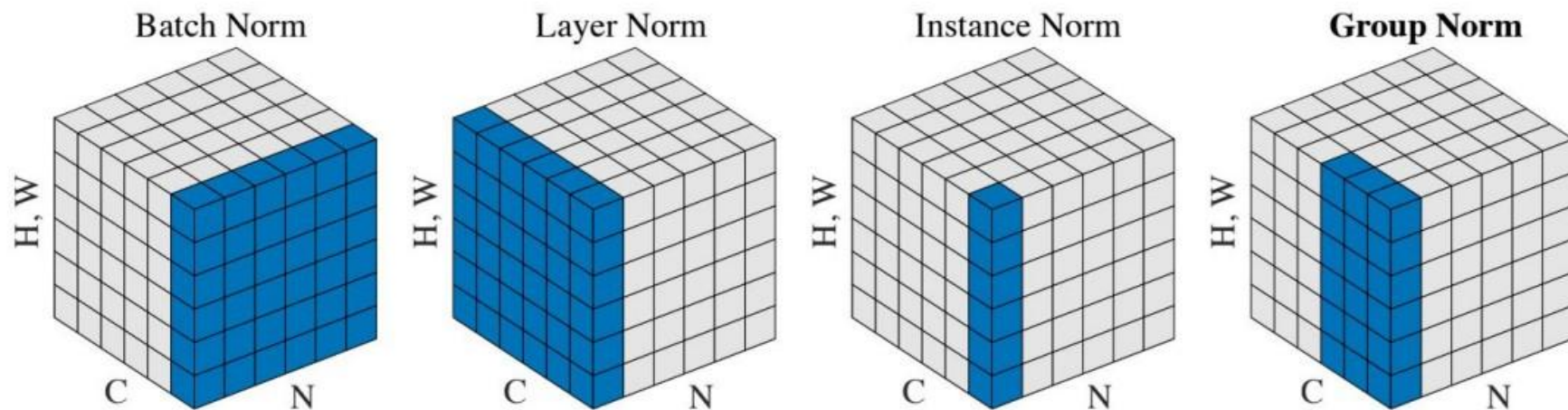
Instance Normalization for
convolutional networks
Same behavior at train / test!

$$\begin{array}{l} \mathbf{x}: \mathbf{N} \times \mathbf{C} \times \mathbf{H} \times \mathbf{W} \\ \text{Normalize} \quad \quad \quad \downarrow \quad \downarrow \\ \boldsymbol{\mu}, \boldsymbol{\sigma}: \mathbf{N} \times \mathbf{C} \times \mathbf{1} \times \mathbf{1} \\ \gamma, \beta: \mathbf{1} \times \mathbf{C} \times \mathbf{1} \times \mathbf{1} \\ \mathbf{y} = \gamma(\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \beta \end{array}$$

So sánh các phương pháp chuẩn hóa



Chuẩn hóa theo nhóm



Tài liệu tham khảo

1. Bài giảng biên soạn dựa trên khóa cs231n của Stanford, bài giảng số 7:

<http://cs231n.stanford.edu>

2. Khởi tạo Xavier:

<https://prateekvjoshi.com/2016/03/29/understanding-xavier-initialization-in-deep-neural-networks/>